

Vehicle Service Management

Congratulations on completing your application definition! This document presents a summary overview of your application as defined in the Pega platform. It encapsulates the key elements of your application's design, highlighting its core functionalities and overall structure.

Please note that this is a high-level view and does not include granular details like data models or sample data. It's an ideal resource for a quick reference, facilitating discussions among team members and stakeholders, and for guiding strategic decisions in your application development process.

Blueprint ID: BP-2418280

Application Context

Urban Fleet Operations is building a Vehicle Service Management application on Pega Platform to manage vehicle servicing for individual customers across its service centres. Customers currently raise service requests through email and phone calls, causing delays, lack of visibility, and inefficient tracking of service history. The application must allow customers to raise vehicle service requests, enable service advisors to inspect vehicles and provide service estimates, capture customer approval before service execution, assign technicians and track service completion, and automatically notify customers when service is completed. The application must support the following business process: 1. Customer submits a Vehicle Service Request with Vehicle ID, Vehicle Model, Vehicle Type, and Issue Description. The required request information must be validated before submission. 2. A Service Advisor performs the vehicle inspection and records Inspection Notes and Condition Rating. The inspection must be completed before the service estimate can be generated. 3. The Service Advisor generates a service estimate by entering Labor Cost and Parts Cost. Total Cost must be calculated as Labor Cost + Parts Cost. Total Cost must be automatically updated and stored in the case. 4. The Customer reviews the Labor Cost, Parts Cost, and Total Cost and approves or rejects the service estimate. The decision must be captured using Approval Status. If approved, the request proceeds to Service Execution. If rejected, the service request is resolved with an appropriate status. 5. During Service Execution, the service request must be automatically routed based on Vehicle Type. Heavy vehicles are routed to HeavyVehicleQueue and all other vehicle types are routed to LightVehicleQueue. 6. A Technician performs the service and the application maintains Assigned Technician, Service Status, and Service Summary. The service assignment must be handled automatically. 7. When the service is completed, the case is resolved and the Customer receives a service completion notification. The notification must contain Case ID, Vehicle ID, Vehicle Model, Service Summary, and Total Cost. 8. The application must maintain a reusable Vehicle data object containing Vehicle ID, Model, and Type. The Vehicle data object must be associated with the Vehicle Service Request case so that vehicle information can be reused across multiple service requests and service history can be tracked. 9. The Vehicle Service Request must have a service-level agreement with a Goal of 2 days and a Deadline of 3 days from case creation. When the Goal is missed, the case should be flagged as approaching its deadline. When the Deadline is missed, the case priority should be automatically increased. No complex escalation path is required. The application should provide a clear end-to-end case lifecycle from vehicle service request creation through Inspection, estimate generation, customer Approval, Service Execution, completion, and customer notification.

Organization name	Industry	Language
Internet Service Provider	Other	English

Users access the application through defined channels, assume specific roles, and trigger workflows that manage how work progresses. Those workflows operate on shared Data Objects to ensure the system stays consistent and up to date.

